

Projet de Programmation Fonctionnelle

Martin Cornillad, Corentin Surier, Florian Hess, Théo Armaghan

27 janvier 2026

Table des matières

1	Introduction	3
2	Organisation du code et détail des implémentations	4
2.1	Iterator.ml	4
2.2	Brick.ml	4
2.3	Barreau.ml	4
2.4	Input.ml	4
2.5	Layout.ml	5
2.6	Config.ml	5
2.7	Collisions.ml	5
2.8	Game.ml	5
2.9	Newtonoid.ml	6
3	Conclusion	6

1 Introduction

L'objectif de ce projet de programmation fonctionnelle était d'implémenter en OCaml un jeu appelé Arkanoid (jeu de casse-briques).

Le jeu, dans sa forme minimale, devait contenir les points suivants. La balle doit pouvoir rebondir sur les bords de l'écran, la raquette et les briques et est soumise à la pesanteur.

La raquette se déplace avec la souris et permet de communiquer des impulsions à la balle.

Les briques touchées disparaissent, et ajoutent une certaine valeur à un score. On perd une vie lorsque la balle tombe et on peut redémarrer le jeu tant qu'on a encore des vies. Au delà des attendus fonctionnels, plusieurs critères de notations étaient donnés.

Dans ce rapport, nous détaillerons ce que nous avons fait pour implémenter ce projet en respectant les critères de notation.

2 Organisation du code et détail des implémentations

Le projet est organisé en plusieurs modules distincts.

Le module `Iterator` contient les flux utilisés dans tout le projet. `Config` centralise tous les paramètres du jeu. `Brick` gère les briques et leur organisation en quad-tree. `Barreau` représente la raquette du joueur. `Input` récupère les entrées souris. `Layout` génère les différentes configurations de niveaux. `Collisions` contient toute la logique de détection et de calcul des rebonds. `Game` implémente la boucle principale du jeu sous forme de flux. Enfin, `Newtonoid` initialise le jeu et gère l’affichage graphique.

2.1 `Iterator.ml`

`Iterator.ml` contient l’implémentation des flux (comme vu en cours) car elle est utilisée en permanence dans le projet. On ne détaillera pas le code qu’elle contient car il ne comporte rien de nouveau par rapport à celui utilisé en `td` et `tp`.

2.2 `Brick.ml`

`Brick.ml` contient l’implémentation d’une brique et du quad-tree permettant de gérer toutes les briques d’un niveau du jeu.

Le type `brick` permet de représenter une brique (taille, position, valeur dans le jeu et couleur).

Le type `bricks` est le quad-tree permettant de gérer toutes les briques d’un niveau. Il possède une structure d’arbre classique comme étudié en `tp`, sauf qu’ils possèdent à chaque noeud une `box` (représente quelle partie de l’espace est gérée par cette branche) ainsi que de 4 branches représentant les 4 parties de l’espace interne à la `box` du noeud (partie sud-est, sud-ouest, etc). Chaque feuille est une liste de briques contenues dans la partie de l’espace représentée par la branche.

La fonction la plus importante est *`build_tree`* car elle permet de construire un quad-tree à partir d’une `box` représentant l’espace initial et une liste de briques. Nous avons ensuite implémenter des fonctions utilitaires permettant de faire les opérations suivantes. On peut vérifier si un quad-tree est vide, à savoir si toutes les briques ont été cassées. On peut aussi supprimer une brique du quad-tree et vérifier si une brique se trouve dans une certaine partie de l’espace (`box`).

2.3 `Barreau.ml`

`Barreau.ml` contient l’implémentation de la raquette du joueur.

Le type `t` est un record contenant les coordonnées de la raquette (`xmin`, `xmax`, `ymin`, `ymax`) ainsi que la position de son centre. La position verticale de la raquette est fixe (définie dans `Config`) et seule sa position horizontale varie selon la souris.

La fonction *`creer`* génère une raquette à partir de la position `x` de la souris, en s’assurant que la raquette reste dans les limites du cadre de jeu.

La fonction *`flux_barre`* crée un flux infini de raquettes en mappant le flux d’input souris vers des raquettes. Ce flux sera ensuite consommé par la boucle principale du jeu.

2.4 `Input.ml`

`Input.ml` nous permet de récupérer dans un flux infini les informations de la souris, à savoir sa position et l’état du clic de la souris (relevé ou enfoncé).

2.5 Layout.ml

Ce module contient différentes implémentations permettant de gérer les différents niveaux du jeu.

Le type `config` contient les informations nécessaires sur un niveau, le nombre de lignes et de colonnes de briques, la taille des briques, l'espacement entre les briques, etc. `value_fn` et `color_fn` permettent de déterminer la couleur et la valeur d'une brique en fonction de sa ligne et sa colonne. Nous avons choisi d'implémenter 3 configurations de niveau différentes (classique, pyramide et échiquier), mais on pourrait en implémenter bien plus.

Les différentes fonctions `make` permettent de créer le niveau à partir de la configuration et notamment de générer le quad-tree à partir de la liste de briques.

2.6 Config.ml

Ce module contient tous les paramètres du jeu, dimensions, nombres de vies, gravité, vitesse maximale etc et nous permet donc d'équilibrer très facilement le jeu à partir de ce fichier sans avoir besoin de fouiller les différents fichiers du projet et de modifier les valeurs une à une.

2.7 Collisions.ml

Ce module permet de gérer les collisions entre les différents éléments du jeu et la balle.

Il y a différentes fonctions permettant de détecter les collisions et de calculer les rebonds de la balle sur les différents objets.

Pour ce qui est de la détection des collisions avec les briques, comme demandé dans le sujet, on utilise la méthode `aabb` qui trouve le point de la brique le plus proche de la balle et si la distance entre ce point et le centre de la balle est plus petit que le rayon, on dit qu'il y a collision. La fonction `find_collinding_brick` met en valeur pourquoi l'utilisation d'un quad-tree est pertinente. En effet, au lieu de tester la collision avec toutes les briques, on teste la collision avec chaque partie de l'espace que représente une branche. S'il la balle ne se trouve pas dans une partie de l'espace, on ne teste pas la collision avec les briques de cette section. On élimine grandement le nombre de briques à vérifier à chaque itération de la boucle du jeu.

Pour les rebonds sur les briques, on ajoute un facteur que nous avons appelé perturbation (aléatoire de $\pm 10\%$ de base, modifiable dans la config) afin d'éviter de pouvoir avoir des rebonds parfaits en vertical si on ne bouge pas la barre au début du jeu notamment. La balle s'accélère en rebondissant sur la barre, mais ne pourra pas dépasser une certaine vitesse maximale.

2.8 Game.ml

Ce module contient la boucle principale sous forme de flux.

Le type `etat` représente l'état du jeu à un instant donné : le quad-tree des briques, le score, le nombre de vies et le niveau actuel.

La fonction `calcul_step` calcule une étape de trajectoire en appliquant la gravité. La fonction `flux_trajectoire` utilise `unfold` pour générer un flux infini de positions et vitesses de la balle soumise à la gravité.

La fonction récursive `flux_jeu` est la boucle principale. À chaque itération, elle teste dans l'ordre : si la balle est tombée (perte de vie ou game over), s'il y a collision avec une brique (mise à jour du score et du quad-tree, passage au niveau suivant si toutes les briques sont cassées), s'il y a collision avec la barre (rebond avec transmission de l'impulsion de la barre), et enfin les collisions avec les murs.

La fonction `run` initialise les flux et lance la boucle de jeu.

2.9 Newtonoid.ml

Newtonoid.ml gère l’affichage graphique et lance le jeu. Les fonctions *draw_brick* et *draw_bricks* permettent de dessiner les briques en parcourant récursivement la structure du quad-tree. *draw_state* dessine l’état complet du jeu (briques, raquette, balle, score et vies).

La fonction *draw* est la boucle d’affichage. Elle consomme le flux de jeu et appelle *draw_state* à chaque itération avec une pause correspondant au dt de la configuration. Quand le flux est vide (game over), elle affiche un message et attend que le joueur appuie sur R pour recommencer ou Q pour quitter.

Le point d’entrée principal initialise le générateur aléatoire et la fenêtre graphique, puis crée les flux d’input et de jeu avant de lancer la boucle de jeu.

3 Conclusion

Ce projet nous a permis de mettre en pratique les concepts de programmation fonctionnelle vus en cours et en TP, notamment les flux infinis et les structures de données immuables.

L’utilisation des flux pour représenter l’état du jeu au cours du temps est particulièrement adaptée à ce type de problème. Chaque élément du jeu (souris, raquette, trajectoire de la balle) est représenté par un flux, et la boucle principale compose ces flux pour produire le flux final des états du jeu.

L’implémentation du quad-tree pour gérer les briques montre l’utilité des structures de données récursives pour optimiser la détection des collisions.

Les principales difficultés rencontrées concernaient la gestion des collisions (calcul des rebonds réalistes).